

电子科技大学
格拉斯哥学院
Glasgow College, UESTC

UESTC 3002: Artificial Intelligence & Machine Learning

Fall Semester 2024-2025

Lab 4 - Week 16

Decision Tree Classifier

Student's Chinese Name	
Student's English Name	
Student's UESTC ID	
Student's UoG ID	

Introduction

The aim of this lab is to provide you with hands-on experience in the complete cycle of supervised learning, as discussed in the lectures. Using the litter and recyclable material dataset you've collected, you will apply a machine learning algorithm, specifically the decision tree classifier, to perform a litter classification task and evaluate the overall performance. Additionally, you will learn how to compute the identification performance for individual classes and represent the results in the form of a confusion matrix.

You are going to finish the next 3 steps using python:

1. Implement a Dataset object from pytorch API for data loading
2. Implement a Machine Learning model and train it
3. Implement the evaluation process of the trained model and draw a confusion matrix

> **Important**
>
> there is an example dataset in the provided material, please replace it with provided full-version dataset.
> Besides, for this machine learning experiment, for convenient we won't split train, test and validation dataset. Both train and evaluation are use the same dataset.

Dataset Object

Please refer to the `dataset.py` file in the material provided in moodle.

The raw data as well as the labels in the dataset are stored in jpg, json etc format. Also the resolution of the images is not uniform. In order to be able to mathematically manipulate the data for later model building, we need to create a unified data interface.

Here we utilize the `torch.utils.data.Dataset` from pytorch package as the data interface. More specifically, Our goal is to create a `Dataset` object which could be indexed by operator `[]`

> **Info**
>
> During the calculation, we use the index of the category to represent itself which starting from 0. For example, 0 refer to `metal`, 1 refer to `food`. Consequently, the size relationship of the index is meaningless. In most cases, the index will than be converted in to so called one-hot vector to emphasise the independency between different categories.

Here is an example of the `Dataset` instance's behaviour:

```
[code]
dataset[0]

[output]
(array([[[159, 146, 137],
        [157, 144, 135],
        [158, 145, 136],
        ...,
        [154, 145, 136],
        [154, 145, 136],
        [154, 145, 136]],
        ...,
        [135, 125, 115],
        [135, 125, 115],
        [135, 125, 115]]], dtype=uint8), array(1, dtype=int64))
```

To implement a `Dataset` object, you need to implemented two methods:

```
def __getitem__(self, index) -> tuple[NDArray[uint8], NDArray[int64]]
#loading a single data by index
def __len__(self) -> int #return the number of total datasamples
```

> **Info**
>
> These two methods are surrounded with `__`, which means they are special methods who will be invoked in specific operation.
> The `__getitem__` method will be invoked when using operation `[]`, e.g. `dataset[0]`
> The `__len__` method will be invoked when using operation `len()`, e.g. `len(dataset)`

Task1

Your task is to implement these two mentioned methods in `dataset.py`.
In the `__getitem__` method, you are going to:

1. Load the exact image from the disk.
2. Crop the image by the bonding box(which is the rectangle we draw in Lab3), And resize it by user defined size: `self.output_size`
3. Convert the cropped image to a numpy array which `dtype=uint8`
4. Convert the annotation to a single int value which represents its category

In the `__len__` method, you are going to return the total number of samples in the dataset

Please refer to the full code in `dataset.py`.

```
    def __getitem__(self, index):
        """
                this method will extract a single data by its index
        """
        id = self.instance_ids[index]
        ann = self.instances[id]
        imgid = ann["image_id"]
        # [x, y , w, h]
        bbox = ann["bbox"]
        img_file = self.coco.loadImgs([imgid])[0]["file_name"]
        img = Image.open(os.path.join(self.data_root, img_file))

        # handle the annotation
        category = ann["category_id"]
        category = np.array(category, dtype=np.int64)

        # crop the image by the bounding box
        _bbox_int = list(map(int, bbox))
        img = img.crop(
            (
                _bbox_int[0],  # x1
                _bbox_int[1],  # y1
                _bbox_int[0] + _bbox_int[2],  # x2 = x1 + w
                _bbox_int[1] + _bbox_int[3],  # y2 = y1 + h
            )
        )
        # resize cropped immg to corresponding size
        img = img.resize(self.output_size, Image.Resampling.BILINEAR)
        if img.mode == "L":
            img = img.convert("RGB")

        # convert image to numpy array
        img = np.array(img)

        assert img.shape[0] == self.output_size[0]
        assert img.shape[1] == self.output_size[1]
        assert img.shape[2] == 3

        return img, category

    def __len__(self):
        """
        the method will reten the number of the dataset
        """
        # NOTE: the total number of instance dataset
        return len(self.instance_ids)
```

Info

1. The `img` is an instance of the PIL.Image.
2. Here we utilizz the `COCO` api from COCO's official repository. It provides a simple API to convert COCO format data annotation to python data structure. You can check the demo from here
3. Use your debug tool to check all the properties defined in the `def __init__(self, data_root, annofile, output_size=(192, 192)):`

Finally, you are going to modify the code under the `if __name__=='__main__'`.
The code under this context will run directly when you run `python3 dataset.py` and print a example data generated by the implemented `Dataset` instance.
The output should looks like:

```
Image size: (192, 192, 3), Image datatype: uint8
Category: 1 Category datatype: int64
```

Attention

Please show your output to GTAs

Creating and training a machine learning model

Please refer to the `mlmodel.py`

Now that we have the loaded data, we are now creating a machine learning model for litter and recyclable material
The overview of the model pipline is as follow:

```
Step 0: Image → Step 1: HOG → Step 2: feature normalization → Step 3:
PCA → Step 4: DecisionTreeClassification
```

About HOG

HOG (**Histogram of Oriented Gradients**) is a feature descriptor used in computer vision(quite popular before deep learning came out) , it will calculate the gradients of small cells in the image and create a representative feature vector for it.

During this process, the trainable steps are step 2, 3 and 4, given in the following section.

> **Trainable process**
>
> When we mention the word **trainable**, it means there are parameters need to be determined during the process.
> In step 2, the parameters are the standard deviation and mean of the data distribution(we assmue the distribution from the training dataset, and fix it during the test phase)
> In step 3, the parameters are the transformation matrixes in the PCA algorithmn
> In step 4, the parameters are the split points, feature selection and tree structure

Task2

The model has been implemented in the given code.. And it contains methods to train itself

The training process could be divided into two stages.
In the first stage the step 2 and step 3 will be trained, which implemented in `def fit_pca`
In the second stage, the step 3 will be trained, which need to be impllemented in `def fit_clf`

your task is to implement the method `def fit_clf`, from line 44, `mlmodel.py`

```
def fit_clf(self, dataset: MyCOCODataset, batch_size, num_workers=0):
        """
        the method should be implemented
        """
        # NOTE: the batc_size here not mean that this model could be trainned in small batches,
        # it's just a way to progressively load the dataset, the trainning is still one pass
        #
        # NOTE: train
        assert self.pca_fitted, "PCA must be fitted before fitting clf"
        feats = []
        cats = []
        print("Extracting features ...")
        for imgs, labels in tqdm(
                DataLoader(
                        dataset,
                        batch_size=batch_size,
                        num_workers=num_workers,
                        collate_fn=self._collate_fn,
                )
        ):
```

```
feats.append(self.pca.transform(self.norm.transform(self.imgs2hogs(imgs)))
)
                cats.append(labels)

        cats = np.concatenate(cats)
        feats = np.concatenate(feats)

        print("Fitting clf ...")
        self.clf.fit(feats, cats)
        print("Fitting clf done")
        # NOTE: student code end here
        self.clf_fitted = True
```

Info

Please refer to the similar training process in `def fit_pca`, note the loaded data from the dataset implemented in task 1 will be stacked, thus there will be one more dimension `batch_size`.
That's why wee need `np.concatenate`

Task3

Now that we have both model and dataset, we need to create a script to run the real training process.
You are going to create a python scripted named `train.py` in the project root folder.
In this script, you are going to create and train the model using our COCO dataset.
After the training finishes, the model should be saved as a python `pickle` object, named `model.pkl`

```
from dataset import MyCOCODataset
from mlmodel import Model
import pickle

if __name__ == "__main__":
    # NOTE: loading the dataset here
    dataset = MyCOCODataset(
        r"data/data_coco",
        r"data/data_coco/annotations.json",
        output_size=(128, 128),
    )

    # NOTE: create the model
    model = Model(
        hog_cell=16,
        hog_block=4,
        n_components=5,
```

```
    )
    print(len(dataset))

    # NOTE: train 2 steps

    model.fit_pca(dataset, batch_size=32, num_workers=0)
    model.fit_clf(dataset, batch_size=32, num_workers=0)

    # NOTE: student code end here
    with open("outputs/model.pkl", "wb") as f:
        pickle.dump(model, f)
```

Info

Since we handle all the logic based on the principle of OOP(Object Oriented Programming). The training script itself is simplified.

Evaluation and Confusion matrix

Evaluation in machine learning is essential to measure a model's effectiveness, identify its strengths and weaknesses, and ensure its predictions are reliable for practical use. Metrics like accuracy, precision, recall, and F1-score help quantify performance and reveal trade-offs, such as balancing false positives and false negatives. A confusion matrix is particularly useful in classification problems, offering a detailed breakdown of predictions by showing true positives, true negatives, false positives, and false negatives. This granular view helps diagnose issues like class imbalance, assess model behavior, and refine decision thresholds, ultimately supporting model selection, fine-tuning, and real-world deployment.

Task4

Refer to the `ml_evaluate.py`
In this task, you are going to evaluate the saved model and draw a confusion matrix for it. You are going to implement the logic of create a confusion matrix. In the missing part of `ml_evaluate.py`

Please follow the code below:

```
def confusion_matrix(predicted, ground_truth, num_classes):
        matrix = np.zeros(shape=(num_classes, num_classes))
        for p, g in np.stack([predicted, ground_truth], axis=1):
                matrix[p, g] += 1
        return matrix
```

⚠ Attention

Please show your printed confusion matrix to the GTAs